

Chapter 7

Transactions

Grouping reads and writes into one unit — and choosing which races the database actually prevents

Source: [claude-skills / ch07-transactions.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part II: Distributed Data

Table of Contents

1. [Core Idea](#)
2. [Framework: ACID](#)
3. [The Isolation Level Ladder](#)
4. [Anomaly-to-Level Mapping](#)
5. [MVCC — Snapshot Isolation Implementation](#)
6. [Two-Phase Locking \(2PL\)](#)
7. [Actual Serial Execution](#)
8. [Serializable Snapshot Isolation \(SSI\)](#)
9. [Mental Models](#)
10. [Anti-Patterns](#)
11. [Worked Example: Doctors On-Call Write Skew](#)
12. [Problems Each Mechanism Solves](#)
13. [Connects To \(Cross-Chapter\)](#)
14. [Key Takeaways](#)
15. [Junior SWE Checkpoints](#)

1. Core Idea

A transaction groups reads and writes into one unit that either **commits entirely** or **aborts entirely**, so the application can safely retry instead of reasoning about partial failure.



Safe retry

Abort on fault → discard partial writes → app retries the whole unit.



Simpler programming

Transactions exist to simplify the model — not a law of nature.



Concurrency control

The real payload: which race conditions exist, which isolation level prevents which.

Weak isolation has caused real money loss. "Use an ACID database" is no protection — most "ACID" databases run **weak isolation by default**. The chapter's job is mapping anomalies to levels and understanding the three roads to serializability: serial execution, 2PL, and SSI.

2. Framework: ACID

Härder & Reuter, 1983. Use as vocabulary, not a guarantee — Kleppmann calls "ACID compliant" mostly a marketing term. **BASE** is even vaguer: effectively "not ACID."

Letter	What it means	What it does <i>not</i> mean
A — Atomicity	Abortability — writes discarded on fault mid-transaction; retry is safe	Not concurrency control. Not "all-or-nothing" across distributed services (that's 2PC/Saga territory)
C — Consistency	Application-defined invariants (e.g. "account balance ≥ 0 ")	A property of the app, not the DB. "The C doesn't really belong in ACID."
I — Isolation	Formally serializability ; in practice usually much weaker	Not what vendor names imply — Oracle "serializable" = snapshot isolation; PG/MySQL "repeatable read" = snapshot isolation
D — Durability	Committed data survives crashes (WAL, fsync, replication)	None absolute — SSDs can violate fsync under power loss; async replication can lose confirmed writes (Ch 5)

Failure mode: assuming the "I" means serializable. Real Bitcoin-exchange losses happened on "ACID" databases at weak isolation. Read the actual isolation level, not the branding.

Real systems: PostgreSQL (MVCC + SSI option), MySQL/InnoDB (MVCC, weak lost-update detection), Oracle ("serializable" = snapshot isolation), DB2 ("repeatable read" = actual serializability), FoundationDB (distributed SSI), VoltDB/H-Store (serial execution).

3. The Isolation Level Ladder

read committed → snapshot isolation → serializable. Each level rules out specific races at increasing cost (locks, version bookkeeping, aborts).

Level	Mechanism	Cost	Failure mode
Read committed	Readers see only committed values; row-level write locks prevent dirty writes	Low — minimal locking	Read skew, lost updates, write skew, phantoms all still possible
Snapshot isolation	MVCC — each transaction reads from a consistent snapshot	Moderate — version storage, visibility rules	Write skew and phantoms still possible; lost-update detection DB-dependent
Serializable	2PL, serial execution, or SSI — transactions behave as if run one-at-a-time	High — blocking, aborts, or single-threaded throughput cap	Deadlocks (2PL), abort storms (SSI), single-core cap (serial execution)

Isolation Level Ladder

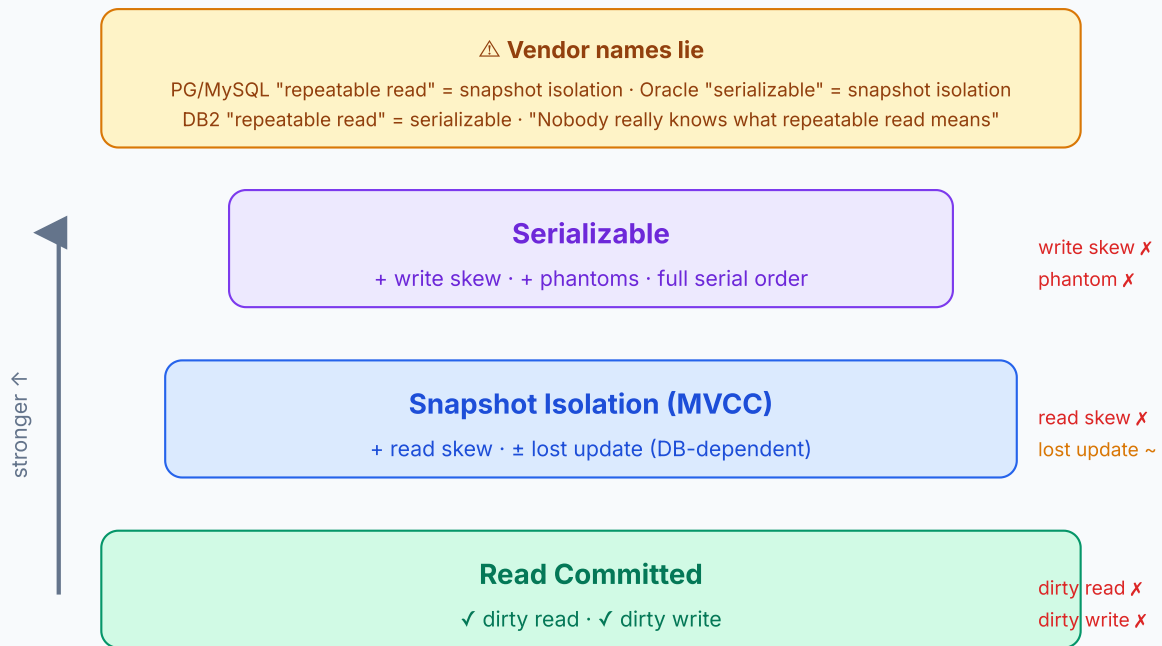


Figure 3.1 — Isolation ladder: each rung adds protection at increasing cost. ~ = DB-dependent.

4. Anomaly-to-Level Mapping

Interview gold: for each anomaly, know the weakest isolation level that prevents it.

Anomaly	What happens	Weakest level that prevents it	Fixes / notes
Dirty read	See uncommitted writes from another transaction	Read committed	Readers get old committed value while write is in flight
Dirty write	Overwrite uncommitted writes (Alice/Bob car sale: Bob wins listing, Alice gets invoice)	Read committed (nearly all implementations)	Row-level write locks held until commit
Read skew (<i>non-repeatable read</i>)	See DB at different points in time (Alice sees \$500 + \$400 = \$900 mid-transfer)	Snapshot isolation	Bites hardest in backups and long analytic/integrity queries
Lost update	Concurrent read-modify-write; later write clobbers earlier (counter 42→43, not 44)	Snapshot isolation (DB-dependent) or explicit locks	Atomic ops (<code>SET value = value + 1</code>), <code>SELECT ... FOR UPDATE</code> , compare-and-set. PostgreSQL repeatable read aborts; MySQL/InnoDB does not. Locks/CAS useless under multi-leader/leaderless (Ch 5)
Write skew	Two txs read same objects, update <i>different</i> objects, each invalidating the other's premise	Serializable	Generalization of lost update. Not caught by snapshot isolation. <code>FOR UPDATE</code> on rows the decision depends on works when rows exist
Phantom	Write changes result of another transaction's search query — worst on <i>absence</i> checks (room free, username untaken)	Serializable	<code>FOR UPDATE</code> has nothing to lock. Needs predicate/index-range locks or serializability. Last resort: materializing conflicts (pre-created lock rows, e.g. room×timeslot table)

Write skew vs phantom: write skew = read → decide → write across *existing* rows (doctors on call).

Phantom = same pattern but the check is for *absence* (no row to lock). FOR UPDATE fixes the former, not the latter.

Read skew: Alice sees inconsistent snapshot mid-transfer

Account A: \$500

(before transfer)

Account B: \$400

(after \$100 moved — Alice read A then B)

Sum = \$900 ≠ \$1000 total

Snapshot isolation fix

Both reads from same snapshot

A: \$500 + B: \$300 = \$800

OR A: \$400 + B: \$400 = \$800

Consistent point-in-time view

Figure 4.1 — Read skew under read committed; snapshot isolation gives a consistent view.

5. MVCC — Snapshot Isolation Implementation

Multi-Version Concurrency Control — how snapshot isolation is built in PostgreSQL, MySQL/InnoDB, Oracle, SQL Server, and others.

When	Default or "repeatable read" in most OLTP databases — the workhorse isolation level.
How	Rows versioned with <code>created_by</code> / <code>deleted_by</code> transaction IDs. Update = delete old version + create new. Visibility rules hide in-progress and later-started transactions' writes.
Why it works	Each reader sees a frozen snapshot; writers create new versions without blocking readers.
Failure mode	Write skew and phantoms slip through — readers and writers don't conflict on different rows.

Mantra: readers never block writers, writers never block readers. The opposite of 2PL. Storage cost: old versions must be garbage-collected (vacuum in PostgreSQL).

MVCC: update = delete + create

v1: balance=500
deleted_by: tx42

v2: balance=400
created_by: tx42

Tx 41 (started before 42)
→ still sees v1 (500)

Visibility: hide uncommitted writes · hide writes from txs that started after reader

Reader sees consistent snapshot without acquiring read locks

Figure 5.1 — MVCC row versioning and visibility rules.

Alternatives (Ch 3): append-only B-trees (CouchDB, Datomic, LMDB) achieve similar copy-on-write semantics without traditional MVCC tuple headers.

6. Two-Phase Locking (2PL)

The classic pessimistic path to serializability. **2PL** ≠ **2PC** (two-phase commit is distributed atomicity, Ch 8–9).

When	Need serializability with general-purpose SQL; acceptable blocking and deadlock risk.
How	Shared locks to read, exclusive locks to write — all held until commit. Acquire phase → release phase (no lock released early).
Why it works	Conflicting operations serialize on locks → equivalent to some serial order.
Failure mode	Readers block writers and vice versa — opposite of MVCC. Unstable tail latency. Deadlocks (auto-detected, one victim aborted).

Predicate locks & phantoms

Predicate locks cover all objects matching a condition, including rows that don't exist yet — kills phantoms but slow. Real systems use **index-range (next-key) locks** — a coarser safe approximation on index entries.

2PL: readers and writers block each other

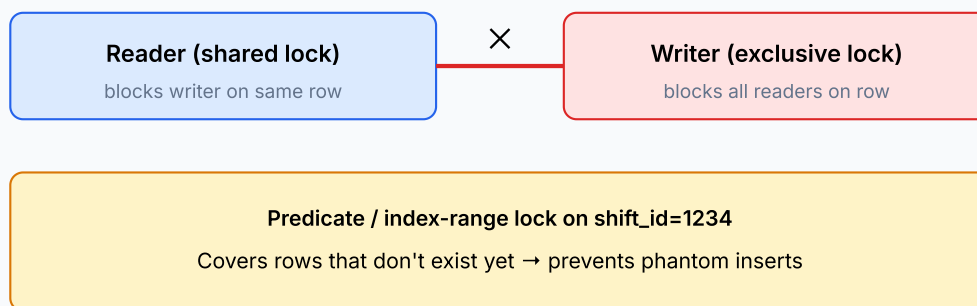


Figure 6.1 — 2PL blocking vs MVCC; index-range locks approximate predicate locks.

Materializing conflicts (last resort): pre-create lock rows (room×timeslot table) so `FOR UPDATE` has something to grab. Ugly — concurrency control leaking into the data model.

7. Actual Serial Execution

Serializable by construction: **one transaction at a time** on one thread. VoltDB/H-Store, Redis (single-threaded), Datomic.

When	Short OLTP transactions, data fits in RAM, throughput capped at one core per partition.
How	Remove all concurrency — no locks, no MVCC, no conflict detection needed.
Why it works	Literally serial → trivially serializable. Viable since ~2007: cheap RAM + fast CPUs.
Failure mode	Requires stored procedures (no interactive round-trips). Cross-partition transactions orders of magnitude slower (Ch 6).

Pessimism to the extreme: serial execution is an exclusive lock on the entire database. Low contention paradise; high contention or long transactions → throughput collapse.

8. Serializable Snapshot Isolation (SSI)

Optimistic serializability: run on a snapshot, detect serialization conflicts at commit, abort losers. PostgreSQL serializable ≥ 9.1 ; FoundationDB distributes it.

When	Mostly low-contention OLTP wanting near-snapshot-isolation performance with serializable guarantees.
How	Detect (a) stale MVCC reads — an ignored write later committed; (b) writes affecting prior reads — index-range "tripwires" that notify, not block.
Why it works	Proceed optimistically; only abort when commit would violate serial order.
Failure mode	High contention → abort storms . Long read-write transactions increase conflict window. Must retry aborts.

vs 2PL

SSI: proceed, abort on conflict. 2PL: block until safe. Low contention → SSI wins.

vs serial execution

SSI allows concurrency; serial execution forbids it entirely.

Retry required

Aborts exist to enable retry — but dedupe (lost commit ack), backoff, transient-only, mind side effects (emails).

SSI conflict detection at commit

(a) Stale read detected

Tx read snapshot; another tx committed write tx ignored

(b) Read-write conflict

Write trips index-range record of prior read → abort loser

Near snapshot-isolation performance · degrades under high contention

First commit wins; second aborts and retries seeing updated state

Figure 8.1 — SSI detects conflicts at commit time instead of blocking during execution.

9. Mental Models

1. Weak isolation is a contract

The DB handles some races, you handle the rest — know exactly which anomalies you've signed up to prevent yourself.

2. Decisions on an outdated premise

Write skew/phantoms = read → decide → write. The premise ("2 doctors on call") can be stale at commit. The DB can't know how you used a query result — only serializability protects the premise.

3. Pessimistic vs optimistic

2PL and serial execution wait (serial execution = exclusive lock on whole DB). SSI proceeds and aborts on conflict. Low contention → optimistic; high contention → pessimistic or commutative atomic ops.

Three roads to serializability: serial execution (in-memory, single-core cap), 2PL (general, slow tails, deadlocks), SSI (optimistic, aborts under contention). Pick based on contention profile and latency requirements.

10. Anti-Patterns

Anti-pattern	Why it fails
Trusting "ACID" branding	Real Bitcoin-exchange losses on "ACID" databases at weak isolation
Read-modify-write in app code (ORM habit)	Silent lost updates — use atomic DB ops (<code>UPDATE SET x = x + 1</code>)
Check-then-act on absence under snapshot isolation	Phantom write skew — bookings, usernames, double-spend checks
Calling single-object CAS "lightweight transactions"	Transactions mean multi-object grouping; CAS is not a transaction
Not retrying aborts (ActiveRecord/Django bubble exception)	Aborts exist to enable retry — but dedupe, backoff, transient-only, mind side effects
Assuming vendor isolation names are standard	PG/MySQL "repeatable read" ≠ DB2 "repeatable read" — read the docs
Locks/CAS under multi-leader replication	Need single up-to-date copy — use commutative ops or sibling merges (Ch 5)

11. Worked Example: Doctors On-Call Write Skew

Shift needs ≥ 1 doctor on call. Alice and Bob are both on call, both sick, both click "go off call" simultaneously.

What each transaction does

```
-- Alice's transaction
BEGIN;
SELECT count(*) FROM doctors
  WHERE on_call = true
  AND shift_id = 1234;
-- sees 2 → passes  $\geq 2$  check

UPDATE doctors SET on_call = false
  WHERE name = 'Alice'
  AND shift_id = 1234;
COMMIT;
-- Both commit → ZERO doctors on call!

-- Bob's transaction
BEGIN;
SELECT count(*) FROM doctors
  WHERE on_call = true
  AND shift_id = 1234;
-- sees 2 → passes  $\geq 2$  check

UPDATE doctors SET on_call = false
  WHERE name = 'Bob'
  AND shift_id = 1234;
COMMIT;
```

Write skew timeline — snapshot isolation misses it

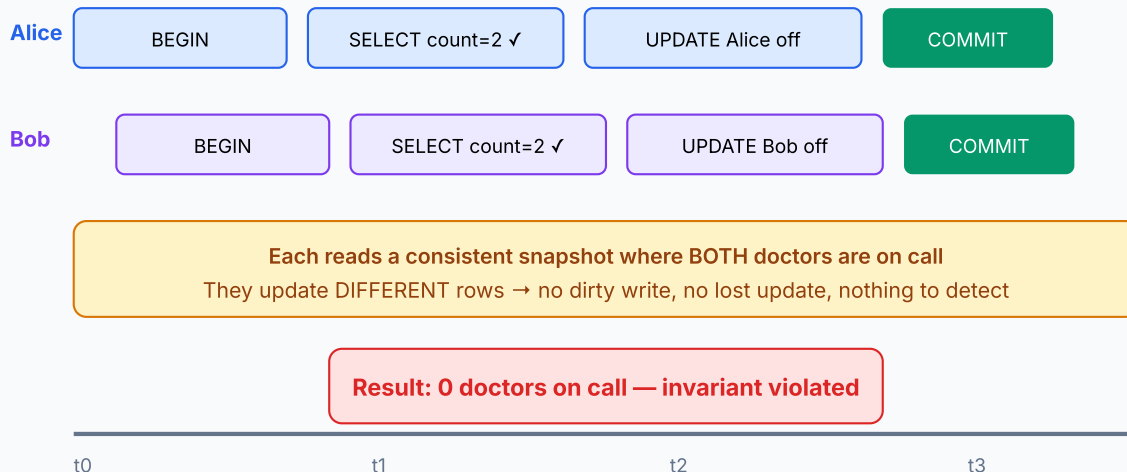


Figure 11.1 — Write skew: serially the second transaction would see count=1 and refuse.

Why snapshot isolation misses it

Each transaction reads a consistent snapshot where both doctors are on call, and they update *different rows* — no dirty write, no lost update, nothing for detection to catch. Serially, the second would have seen count = 1 and refused; the anomaly exists only under concurrency.

Fixes

Approach	How it works here
Serializable isolation (SSI)	Each UPDATE trips index-range record of the other's read on shift 1234; first commits, second aborts and retries seeing count = 1
2PL	Second SELECT blocks until first commits; then sees count = 1
SELECT ... FOR UPDATE	Works <i>here</i> because on-call rows exist to lock — unlike phantom variants
DB constraint	"At least one on call" isn't expressible as a standard constraint — needs app logic or serializability

That's write skew: read-decide-write across objects where each transaction's decision is based on a premise that the other transaction invalidates — without touching the same row.

12. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Atomic read-modify-write (locks, Lua scripts, sorted sets)	Race on shared counter under concurrent access	Rate limiter (Redis counter races)
Non-atomic multi-store commit made safe (pending→uploaded flag)	Multi-store write isn't wrapped in single DB transaction; partial completion must be representable	Google Drive upload (metadata <code>pending</code> , flipped on S3 callback)
Offset + downstream write committed atomically	At-least-once delivery must not double-count high-value aggregates	Ad click event aggregation ("a few percent = millions of dollars")
Lost-update walkthrough → <code>SELECT FOR UPDATE</code> , optimistic version column, <code>CHECK</code> , idempotency-key unique constraint	Double-booking / double-submission under concurrent requests	Hotel reservation system
Double-entry ledger on ACID SQL + idempotency-key unique constraint	Money must never be created or destroyed; retries must not double-move it	Payment system
2PC → TC/C → Saga (distributed-transaction spectrum)	Atomicity across services when no single DB transaction spans them	Digital wallet
WAL + transactional mapping-table update during compaction	Merging small objects can't leave object-to-location mapping inconsistent mid-merge	S3-like object storage (secondary)

13. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 3 Storage	WAL underpins atomicity/durability; append-only B-trees (CouchDB, Datomic, LMDB) as alternative MVCC	"Atomicity = WAL + abort; durability = fsync + replication."
Ch 5 Replication	Lost updates across replicas, LWW, siblings/merges; durability via replication; locks/CAS need single up-to-date copy	"Multi-leader breaks SELECT FOR UPDATE — use commutative ops or merges."
Ch 6 Partitioning	Why serial execution partitions; cross-partition transactions orders of magnitude slower	"Serializable per partition ≠ global serializability."
Ch 8–9 Distributed	Distributed faults; distributed transactions, 2PC (≠ 2PL), linearizability (CAP's "consistency")	"2PL is single-node serializability; 2PC is cross-service atomicity."

14. Key Takeaways

1. **Atomicity = abortability; Consistency is your job;** "ACID" says almost nothing about actual isolation level.
2. **The ladder:** read committed kills dirty reads/writes; snapshot isolation (MVCC) adds read-skew protection; only serializable kills write skew and phantoms (lost updates: sometimes detected at snapshot isolation, DB-dependent).
3. **Vendor names lie:** PG/MySQL "repeatable read" and Oracle "serializable" = snapshot isolation; DB2 "repeatable read" = serializable.
4. **Write skew = read-decide-write across objects;** phantoms = write skew on absence checks. `FOR UPDATE` fixes the former, not the latter.
5. **Three roads to serializability:** serial execution (in-memory, single-core cap), 2PL (general, slow tails, deadlocks), SSI (optimistic, aborts under contention).
6. **Locks/CAS need a single up-to-date copy** — they break under multi-leader/leaderless replication (Ch 5).

15. Junior SWE Checkpoints

Q1: What does atomicity actually mean in ACID?

Abortability — if a fault occurs mid-transaction, all writes are discarded and the app can safely retry. Not concurrency control. Not distributed atomicity (that's 2PC/Saga).

Q2: User A and B both read counter=42, both increment, both write 43. Expected 44. What anomaly?

Lost update. Fix: atomic `UPDATE SET x = x + 1`, `SELECT FOR UPDATE`, or optimistic version column. PostgreSQL repeatable read aborts; MySQL/InnoDB does not.

Q3: Two doctors go off call simultaneously; zero remain. Why doesn't snapshot isolation catch it?

Write skew. Each reads a snapshot with count=2, updates different rows. No dirty write, no lost update on same row. Needs serializable isolation or `FOR UPDATE` on the on-call rows.

Q4: Booking system checks "no row for room 42 on Tuesday" then inserts. Two users book simultaneously. What anomaly?

Phantom. Absence check — `FOR UPDATE` has nothing to lock. Needs predicate/index-range locks (2PL) or SSI, or materialize conflict rows.

Q5: What's the difference between 2PL and 2PC?

2PL = two-phase *locking* for single-node serializability (shared/exclusive locks held to commit). **2PC** = two-phase *commit* for distributed atomicity across services (Ch 8–9). Completely different mechanisms.

Q6: When does SSI beat 2PL? When does 2PL beat SSI?

SSI wins at low contention — near snapshot-isolation performance, abort only on actual conflict. **2PL wins** at high contention — blocks instead of abort storms; or use commutative atomic ops.

Q7: Why doesn't "we use PostgreSQL, it's ACID" protect against write skew?

PostgreSQL default is **read committed**; "repeatable read" is snapshot isolation, not serializable. Must explicitly set `SERIALIZABLE` (SSI) or use `FOR UPDATE` / app-level constraints.

Glossary

Term	Definition
Atomicity	Abortability — all-or-nothing within one transaction on one node
Snapshot isolation	Each transaction reads from a consistent point-in-time snapshot (MVCC)
MVCC	Multi-version concurrency control — row versions with visibility rules
Read skew	Seeing DB at different points in time within one transaction
Lost update	Concurrent read-modify-write; later write clobbers earlier
Write skew	Transactions read same data, write different rows, invalidate each other's premise
Phantom	Write changes the result set of another transaction's query
2PL	Two-phase locking — shared/exclusive locks held until commit
Predicate lock	Lock on all rows matching a condition, including not-yet-existing rows
SSI	Serializable snapshot isolation — optimistic conflict detection at commit
Materializing conflicts	Pre-creating lock rows so FOR UPDATE can prevent phantoms
Serial execution	One transaction at a time — trivially serializable, single-core cap